

JobPilot - Smart Job Matcher and Resume Builder

BAX-423 Final Project Technical Brief | Hanzhi Wang | Deployment: local package is ready for public hosting; add the hosted URL in Canvas after deployment | Source: submitted ZIP and course repository link

1. System Overview and Architecture

JobPilot is a Streamlit job matching and resume tailoring application. It uses an offline dataset of 31,047 deduplicated postings from Kaggle Techmap and JSearch API rows. The architecture is: ingestion and schema normalization -> cleaned CSV snapshot -> MiniLM sentence embeddings -> FAISS retrieval -> hard filters -> multi-stage ranking -> Streamlit recommendations, feedback, CSV export, analytics, and resume draft generation.

2. Pipeline Design

Data ingestion is split between a streaming Techmap sampler and JSearch API collection. The Techmap sampler includes robust scalar/location cleaning so nested schema.org dictionaries do not leak into CSV strings. Job records are normalized to title, company, location, description, skills, seniority, employment type, salary, source, apply URL, and clean embedding text. At query time, the app accepts PDF, DOCX, TXT, pasted resume text, or structured profile text. The parser extracts name, contact fields, target roles, location, salary, skills, years, candidate level, max acceptable required years, dealbreakers, industries, education, projects, and raw resume text.

3. BAX-423 Techniques

Technique 1: dense embeddings plus FAISS approximate nearest-neighbor retrieval. Jobs are embedded with sentence-transformers/all-MiniLM-L6-v2 and searched using a FAISS inner-product index. Technique 2: multi-stage ranking and filtering. Candidate jobs pass salary, seniority, internship, contract, unpaid, location, role-family, experience-years, and low-quality-posting filters, then receive role, target-fit, skill, experience, location, salary, quality, source, and feedback scores. Technique 3: adaptive feedback. Save, apply-later, and not-interested actions update preferred and avoided terms that affect the next ranking pass.

4. Matching Quality and Explanations

Role fit is stronger than generic semantic similarity. The recommender classifies target and job families such as analytics, data science, ML engineering, MLOps/data engineering, software, finance, and product. It avoids unrelated developer roles unless requested, filters senior roles for entry candidates, extracts required years from job text, annualizes salary, penalizes missing salary, and filters placement/training/job-seeker style postings using general text signals. Each recommendation includes why_matched plus diagnostic fields including salary_annual_min, salary_annual_max, required_years, job_quality_score, target_fit_score, and component scores.

5. App Capabilities

The UI is a six-step flow: upload/paste resume, review extracted profile, confirm preferences and dealbreakers, run matching, review recommendations with feedback, and generate a tailored resume draft. Resume generation uses OpenRouter when OPENROUTER_API_KEY is available and falls back to a conservative local Markdown draft. The prompt instructs the model to preserve original facts and avoid invented employers, schools, dates, metrics, or skills. Users can download ranked jobs as CSV with diagnostic scores.

6. Limitations

Kaggle base data is historical, and some postings omit salary or visa information, so salary and sponsorship filters are best-effort when fields are missing. Some source metadata is noisy, so internship and seniority filtering relies on title/description evidence rather than blindly trusting employment_type. A public hosting URL and repository URL should be provided with the Canvas submission after deployment.

7. Persona Test Results

Persona	Top 10	Senior	Contract	Result
Aisha - ML Pivot	10	0	2	PASS
Marcus - New Grad Analytics	10	0	0	PASS
Priya - ML Platform	10	8	2	REVIEW

Submission contents: code/, data/ with offline CSV and embedding artifacts, brief.pdf, prompts.md, README.md, and no real API keys.